

DEPARTMENT OF COMPUTER SCIENCE • SINGLE SUBJECT SERIES

CS-301

OPERATING SYSTEMS MASTER TEXTBOOK

Comprehensive 23-Page University Lecture Notes, Theory Foundations &
Solved Exam Question Bank

OFFICIAL 23-PAGE SINGLE SUBJECT EDITION • VERIFIED ACADEMIC MASTERPIECE

CURRICULAR SCOPE & CORE MODULES INCLUDED:

- Process Lifecycle, Process Control Block (PCB) & Context Switching
- CPU Scheduling Algorithms: FCFS, SJF, Round Robin, Priority
- Process Synchronization: Critical Section, Mutex, Semaphores
- Deadlocks: Necessary Conditions, Bankers Algorithm, Prevention
- Virtual Memory: Paging, Page Replacement (FIFO, LRU, Optimal)

AUTHOR & PUBLICATION METADATA

Author / Curator: Asad Usman (CS Expert)

Curriculum Year: 2026 - 2027 Academic Session

Department: Department of Computer Science

Verification: Peer-Reviewed CS Hub

Academic Series: Undergraduate Subject Handbook • Edition 2026 Status: Full Academic Release

SYLLABUS**CS-301: Operating Systems**

Hardware abstraction, process scheduling, synchronization primitives, memory virtualization, deadlocks, and file systems.

COURSE OVERVIEW

Kernel architecture, process lifecycle, thread synchronization, CPU scheduling algorithms, deadlock prevention, virtual memory, demand paging, and file systems.

1.0 Accredited Learning Outcomes**Outcome 1**

Trace context switching, interrupt handlers, and system call traps between User Mode and Kernel Mode.

Outcome 2

Implement synchronization primitives (mutexes, semaphores, conditional variables) to eliminate race conditions.

Outcome 3

Apply CPU scheduling algorithms (Round Robin, SRTF, Multilevel Feedback Queues) and calculate turnaround times.

Outcome 4

Analyze virtual memory translation via page tables, TLBs, and page replacement heuristics.

NAVIGATION

Comprehensive Table of Contents

Detailed Index across all 23 Pages of CS-301

Subject Module / Chapter Title	Page
Course Overview & Program Learning Outcomes	Page 2
Detailed Table of Contents	Page 3
Chapter 1: Foundational Theory & Architectural Primitives	Pages 4 - 6
Chapter 2: Structural Mechanics & Idiomatic Implementation	Pages 7 - 9
Chapter 3: Advanced Optimization & Complexity Analysis	Pages 10 - 12
Chapter 4: Complete University Slide Deck Notes (Part 1)	Pages 13 - 14
Chapter 5: Complete University Slide Deck Notes (Part 2)	Pages 15 - 16
Chapter 6: Midterm & Final Solved Examination Bank	Pages 17 - 18
Chapter 7: Semester Capstone Lab Specification & Rubric	Pages 19 - 20
Chapter 8: FAANG Technical Interview Preparation Guide	Pages 21 - 22
Chapter 9: Subject Academic Lexicon & Authoritative Bibliography	Page 23

CHAPTER 1**Processes, Threads & Kernel Dual-Mode Protection**

Context Switching, PCB State Transitions, and System Calls

An operating system is the core resource manager. Hardware enforcement relies on dual-mode CPU operation: Ring 3 (User Mode) restricts access to hardware registers, while Ring 0 (Kernel Mode) grants unlimited privileges. System calls (syscall/sysenter) execute controlled transitions via software interrupts.

Theoretical Foundation 1.1

Process Control Block (PCB): Kernel data structure storing PID, Program Counter, CPU registers, memory mappings, and open file descriptors.

Theoretical Foundation 1.2

Context Switch: Saving state of the running process to its PCB and loading the state of the next scheduled process.

Theoretical Foundation 1.3

Processes vs Threads: Processes have isolated virtual address spaces; threads within the same process share code, data, and heap, but maintain private stacks.

SOURCE LISTING: POSIX Forking, Child Process Execution & Exit Status Reaping (C)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        return 1;
    } else if (pid == 0) {
        // Child Process
        printf("[Child PID: %d] Executing child task...\n", getpid());
        _exit(42); // Clean child exit
    } else {
        // Parent Process
        int status;
        printf("[Parent PID: %d] Waiting for child PID %d...\n", getpid(), pid);
        waitpid(pid, &status, 0); // Reaping prevents zombie process
        if (WIFEXITED(status)) {
            printf("[Parent] Child exited cleanly with status code: %d\n", WEXITSTATUS(status));
        }
    }
    return 0;
}
```

TECHNICAL ANALYSIS

Demonstrates fork() copying process state and waitpid() preventing zombie processes in Linux.

SAMPLE EXAM QUESTION: Midterm Classic

How many total processes are created when the code snippet: `fork(); fork(); fork();` executes?

SOLUTION & KEY TAKEAWAY

Each call to `fork()` doubles the number of active processes. Starting with 1 parent process:

After 1st fork: $2^1 = 2$ processes.

After 2nd fork: $2^2 = 4$ processes.

After 3rd fork: $2^3 = 8$ processes.

Total processes running = 8. Total NEW child processes created = $8 - 1 = 7$.

CHAPTER 2**Concurrency, Race Conditions & Synchronization Primitives**

Semaphores, Mutexes, Monitors & The Dining Philosophers Problem

When multiple concurrent threads access shared mutable state without synchronization, execution outcome depends on arbitrary instruction interleaving, producing non-deterministic race conditions.

Theoretical Foundation 2.1

Critical Section Problem: Protocol must satisfy Mutual Exclusion, Progress, and Bounded Waiting.

Theoretical Foundation 2.2

Mutex: A locking mechanism where only the thread that locked the mutex can unlock it (ownership semantics).

Theoretical Foundation 2.3

Counting Semaphore: An integer variable accessed via atomic `wait()` / `P()` and `signal()` / `V()` operations.

Theoretical Foundation 2.4

Deadlock Four Conditions (Coffman): Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait.

SYSTEM ARCHITECTURE

Detailed architectural trace.

SAMPLE EXAM QUESTION: Final Exam Mastery

State the four Coffman conditions for deadlocks and explain Banker algorithm safety check.

SOLUTION & KEY TAKEAWAY

The four conditions are Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait. The Banker algorithm tests safety before granting resource requests: it simulates allocation and checks if there exists a sequence $\langle P_1, P_2, \dots, P_n \rangle$ where each process can finish using currently available resources plus resources freed by previously finished processes.

LECTURE NOTES

CS-301 Slide Deck: Core Modules 1 & 2

Classroom Lecture Companion

SLIDE 1: MODULE 1: PROCESSES, THREADS & CONTEXT SWITCHING

- Dual-mode operation: User mode vs Kernel mode via hardware trap instructions
- Process address space: Text (code), Data, BSS, Heap (grows up), Stack (grows down)
- PCB contents: PID, program counter, register values, memory limits, open file descriptors
- Kernel threads vs User-level threads: context switch overhead and blocking semantics

SLIDE 2: MODULE 2: CPU SCHEDULING ALGORITHMS

- Preemptive vs Non-preemptive scheduling strategies
- First-Come First-Served (FCFS) and the Convoy Effect
- Shortest Job First (SJF) is provably optimal for minimum average waiting time
- Round Robin (RR) with time quantum q : trade-off between responsiveness and context-switch cost

SLIDE 3: MODULE 3: PROCESS SYNCHRONIZATION & CONCURRENCY

- Critical Section Requirements: Mutual Exclusion, Progress, Bounded Waiting
- Hardware atomic instructions: Test-and-Set, Compare-and-Swap
- Counting Semaphores vs Binary Semaphores (Mutexes)
- Classic problems: Producer-Consumer (bounded buffer), Readers-Writers, Dining Philosophers

SLIDE 4: MODULE 4: DEADLOCKS & BANKERS ALGORITHM

- Four Coffman Conditions: Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait
- Deadlock prevention eliminates at least one of the 4 Coffman conditions
- Bankers Algorithm: Safe state check using Available vector, Allocation, Max, and Need matrices
- Resource Allocation Graph (RAG): Cycle with single-instance resources implies deadlock

KEY EXAM HIGHLIGHTS FROM LECTURES

1. Always construct Gantt charts carefully for Round Robin and calculate exact Average Waiting & Turnaround times.
2. In Bankers Algorithm, write down the Need Matrix (Max - Allocation) clearly before testing safe state vectors.
3. Distinguish internal vs external fragmentation with concrete memory diagrams.

LAB & PRACTICUM CHECKLIST

Project Option 1: Simulated Multi-Level Feedback Queue (MLFQ) CPU Scheduler

Project Option 2: Producer-Consumer Concurrent Simulation with POSIX Threads & Mutex

Project Option 3: Simple Unix Shell Implementation with Fork, Exec, and Pipe redirection

CHAPTER 6**CS-301 Midterm Examination Solved Bank**

Step-by-step Model Solutions

MIDTERM PROBLEM [Final Exam Mastery]

Explain Virtual Memory Translation using a 2-level page table and describe the role of the TLB.

OFFICIAL MODEL SOLUTION

Virtual addresses are divided into Page Directory Index, Page Table Index, and Offset. The CPU first checks the TLB (Translation Lookaside Buffer) associative cache. On a TLB hit, physical frame number is retrieved in $< 1\text{ns}$. On a TLB miss, a hardware page table walk traverses memory to locate the Page Table Entry, updates the TLB, and restarts the instruction.

CHAPTER 6 (CONT.)**CS-301 Final Examination Mastery**

Comprehensive Rubrics

EXAMINATION PROTOCOL & SCORING RUBRICS

1. Allocate time proportionally to marks.
2. Write formal preconditions and loop invariants.
3. Show scratch work for memory diagrams.
4. Clearly state time and auxiliary space complexity.

CHAPTER 7

CS-301 Semester Capstone Lab Specification

Hands-on Production Implementation

GRADING CRITERIA & DEMO EXPECTATIONS

- Correctness & Completeness: 40%
- Code Architecture & Clean Principles: 25%
- Memory Safety & Optimization: 20%
- Technical Documentation & Video Demo: 15%

CHAPTER 8**CS-301 Technical Interview Preparation**

FAANG & High-Growth Tech Interview Questions

INTERVIEW SCENARIO: Unix Production Systems Troubleshooting

What is the exact distinction between a Zombie process and an Orphan process in Linux?

OPTIMAL ARCHITECTURAL SOLUTION

An orphan process is a child whose parent terminated without waiting for it; the Linux init process (PID 1) automatically adopts orphan processes and reaps their exit status. A zombie process is a process that has completed execution via `exit()`, but its parent has not yet called `wait()` to read its exit code; its PCB remains in the process table, consuming PID slots.

INTERVIEW CHECKLIST

1. Clarify constraints and expected volume.
2. Explain algorithmic intuition before typing.
3. Identify space-time trade-offs.
4. Walk through an edge case manually.

CHAPTER 9

CS-301 Lexicon & Authoritative References

Glossary of Formal Terms and Literature

Key Term	Rigorous Academic Definition
Context Switch	The switching of the CPU from one process or thread to another, involving state save and restore.
Thrashing	A state where the operating system spends more time swapping pages in and out of virtual memory than executing ac

- RECOMMENDED TEXTBOOKS
1. Operating System Concepts ("Dinosaur Book") - Silberschatz, Galvin, Gagne

2. Modern Operating Systems - Andrew S. Tanenbaum

3. Operating Systems: Three Easy Pieces (OSTEP) - Remzi & Andrea Arpaci-Dusseau